

AWS RDS Masterclass

Complete Guide to Managed Relational Databases

Database Engines, High Availability, Performance Optimization, Security, and Production Deployment

Table of Contents

- 1. Introduction to AWS RDS
- 2. RDS Fundamentals and Architecture
- 3. Database Engines and Selection
- 4. Instance Types, Sizing and Performance
- 5. Storage, IOPS and I/O Performance
- 6. Backup, Recovery and Disaster Recovery
- 7. Security, Encryption and Network
- 8. High Availability and Multi-AZ Deployment
- 9. Read Replicas and Scaling Reads
- 10. Parameter Groups, Options and Configuration
- 11. Monitoring, Logging and Performance Insights
- 12. Cost Optimization and Analysis
- 13. Amazon Aurora Advanced Features
- 14. Production Best Practices
- 15. Troubleshooting Guide

1. Introduction to AWS RDS

AWS Relational Database Service (RDS) is a managed database service that handles infrastructure provisioning, patching, backups, and replication. RDS supports six popular database engines: MySQL, PostgreSQL, MariaDB, Oracle Database, SQL Server, and Amazon Aurora. By offloading database administration tasks to AWS, teams focus on application development rather than database operations.

Key Concept: RDS is "managed" - AWS handles provisioning, patching, backups, failover, and replication. You focus on data modeling, optimization, and application logic. This dramatically reduces operational overhead while ensuring high availability and durability.

Why RDS Matters for DevOps & SREs

- **Automated Administration:** AWS handles provisioning, patching, backups, replication, and failover automatically
- **High Availability:** Multi-AZ deployments provide automatic failover with RPO approaching zero
- **Scalability:** Read replicas distribute read traffic, primary scaling with parameter optimization
- **Backup & Recovery:** Automated daily backups with point-in-time recovery up to 35 days
- **Security:** Encryption at rest and in transit, VPC isolation, IAM database authentication
- **Performance Monitoring:** Enhanced Monitoring, Performance Insights, and CloudWatch integration
- **Cost Efficiency:** Pay-as-you-go pricing, no capacity over-provisioning, automatic scaling options
- **Compliance:** Supports compliance requirements (HIPAA, PCI-DSS, SOC 2) with audit logging

RDS Database Engines Comparison

Engine	License	Performance	Scaling	Best For	Typical Use
MySQL	Open Source	Fast reads, good OLTP	Read replicas, no sharding built-in	Web applications, content management	WordPress, Drupal, custom web apps

PostgreSQL	Open Source	Complex queries, ACID compliant	Read replicas, logical replication	Complex data models, JSON, geospatial	Data warehousing, PostGIS, complex logic
MariaDB	Open Source	MySQL compatible, faster	Read replicas, Galera clustering	MySQL replacement with improvements	High-traffic web applications
Oracle	Proprietary	Enterprise features, high performance	Data Guard, many advanced features	Enterprise applications, legacy systems	ERP systems, banking, insurance
SQL Server	Proprietary	Windows ecosystem, BI integration	Always On, mirroring, replication	.NET applications, Windows integration	Enterprise .NET apps, Windows shops
Aurora MySQL	Proprietary (AWS)	High performance (3x MySQL)	Auto-scaling, Aurora Serverless	High-scale OLTP, Serverless needs	SaaS platforms, mobile backends
Aurora PostgreSQL	Proprietary (AWS)	High performance (3x PostgreSQL)	Auto-scaling, serverless read replicas	Complex queries at scale	Analytics, complex data models at scale

RDS vs Self-Managed Databases

RDS (Managed)

- AWS handles patching, backups, replication
- Automatic failover (Multi-AZ)
- Minimal operational overhead
- Built-in monitoring and alerting
- Point-in-time recovery (35 days)
- Higher cost per transaction
- Less customization flexibility

Self-Managed (EC2)

- Full control and customization
- Lower per-unit cost at scale
- Custom configurations possible
- Must manage patching, backups, replication
- Team responsible for 24/7 monitoring
- Complex HA setup required
- Higher operational complexity

💡 **Pro Tip:** For most workloads, RDS is optimal - costs increase for small deployments but decrease for production systems. Self-managed databases only make sense for highly specialized requirements (e.g., custom database versions, specific kernel tuning) or massive scale (1000+ cores).

2. RDS Fundamentals and Architecture

Understanding RDS architecture, components, and deployment models is essential for designing reliable database infrastructure. RDS provides several deployment options with different characteristics, performance levels, and cost profiles.

RDS Architecture and Components

Component	Description	Purpose	Configuration
DB Instance	Single database server with allocated compute and storage	Primary database server for read/write operations	Instance class (db.t3.micro to db.x1e.32xlarge), storage 20GB-65TB
DB Cluster	Group of DB instances (Aurora only) with shared storage	High availability with automatic failover	Primary + multiple read replicas, shared storage layer
Subnet Group	Collection of VPC subnets for RDS placement	Ensures Multi-AZ deployment across availability zones	Minimum 2 subnets in different AZs
Parameter Group	Template for database configuration parameters	Controls database behavior (max_connections, innodb_buffer_pool_size)	Engine-specific, can be modified with restart
Option Group	Engine-specific features and extensions	Adds features (Oracle Advanced Security, SQL Server Analysis Services)	Optional, applied per instance

Security Group	Firewall rules for database access	Controls network access to database port	Inbound rules for port 3306 (MySQL), 5432 (PostgreSQL), etc
Backup Storage	Automated and manual snapshots	Point-in-time recovery, disaster recovery	Automated daily backups (35-day retention), manual snapshots unlimited
Read Replica	Read-only copy of database in same or different region	Scale read traffic, disaster recovery across regions	Same or different instance class, can be in different region

RDS Deployment Models

Model	HA	RPO	RTO	Cost	Best For
Single-AZ	None (no failover)	N/A	Manual recovery (hours)	Base cost only	Development, testing, non-critical workloads
Multi-AZ	Automatic failover (60-120 sec)	<1 second (synchronous replication)	60-120 seconds	2x cost (standby replica)	Production workloads, critical applications
Read Replicas (Same Region)	Only for reads (manual failover)	<100ms typically	Manual promotion (seconds)	Per replica instance cost	Scale read traffic, reporting databases
Read Replicas (Cross-Region)	DR across regions	<1 second typically	Manual promotion + DNS update	Replica + data transfer cost	Disaster recovery, global read distribution
Aurora with Replicas	Auto failover, auto scaling	RPO ~0s, distributed storage	30 seconds or less	Per vCPU + storage (pay-as-you-go)	High-scale OLTP, serverless applications

⚠ Critical: Single-AZ databases have zero HA. If instance fails or AZ goes down, manual intervention required. For production, always use Multi-AZ or read replicas. RPO/RTO of Multi-AZ approaches zero - standby is continuously synchronized.

3. Database Engines and Selection

RDS supports six database engines, each with different strengths, performance characteristics, and operational requirements. Choosing the right engine significantly impacts application performance, maintainability, and cost.

MySQL vs PostgreSQL Detailed Comparison

Aspect	MySQL	PostgreSQL	Recommendation
Data Types	Basic: INT, VARCHAR, DATE, DECIMAL. No native JSON support (added recently)	Rich: All MySQL types + JSON, JSONB, Arrays, UUID, HStore, PostGIS	Complex data → PostgreSQL. Simple web apps → MySQL
Transactions	ACID (with InnoDB), Row-level locking, good concurrency	Full ACID, excellent concurrency, MVCC, serializable isolation	PostgreSQL for complex transactions, high concurrency
Indexes	B-tree, Hash, Full-text. No partial indexes or expression indexes	B-tree, Hash, GIN, GiST, BRIN, Partial, Expression, CONCURRENTLY	PostgreSQL for complex query optimization
Query Performance	Fast for simple queries, simple optimizer	Excellent for complex queries, sophisticated query planner	PostgreSQL for complex analytical queries
Replication	Asynchronous binary replication, Group Replication	Streaming replication, logical replication, slots	PostgreSQL logical replication for selective replication
Learning Curve	Easier for SQL learners, more forgiving	Stricter SQL standard compliance, steeper curve	MySQL for teams new to databases

Ecosystem	Larger ecosystem (WordPress, Drupal), widespread hosting support	Growing ecosystem, strong in data warehousing and analytics	MySQL for widely-used applications
Cost (RDS)	Similar per instance, lower per operation for simple queries	Similar per instance, more efficient for complex queries	Similar - choose based on requirements

Aurora - The AWS-Native Database Engine

Aurora is AWS's proprietary database engine built from scratch for cloud, compatible with MySQL and PostgreSQL wire protocols. Aurora provides superior performance, availability, and cost compared to open-source engines.

Aurora Architecture (MySQL and PostgreSQL compatible)

Component	Traditional RDS	Aurora	Advantage
Storage	Attached EBS volume, single instance manages	Distributed storage (6 replicas across 3 AZs), managed separately	Storage scaling independent of compute, durability
Read Replicas	Separate instances, asynchronous replication	Share storage, synchronous replication, <100ms lag	Instant read scaling, lower RPO
Failover	60-120 seconds with Multi-AZ, data resync	30 seconds or less, storage already available to replica	Faster RTO, zero data loss
Scaling	Manual instance change or read replicas (slow)	Auto-scaling read replicas, adjust in minutes	Automatic horizontal scaling, cost efficient
Backups	Automated snapshots, point-in-time recovery	Continuous backup to S3, 35-day retention	Minimal backup overhead, instant recovery
Performance	Good, standard database performance	3x MySQL, 2x PostgreSQL throughput, lower latency	Engineered for cloud, optimized I/O

i Information: Aurora is generally recommended for new applications at scale. Performance advantages justify any differences from open-source. Aurora Serverless adds auto-scaling compute, ideal for variable workloads. For legacy applications, maintain MySQL/PostgreSQL compatibility layer.

4. Instance Types, Sizing and Performance

RDS instance types range from db.t3.micro (1 vCPU, 1GB RAM) to db.x1e.32xlarge (128 vCPUs, 3.95TB RAM). Choosing correct instance size is critical - too small causes performance issues, too large wastes money. Proper sizing considers CPU, memory, network, and I/O requirements.

RDS Instance Family Types

Family	Processor	vCPU Range	Memory Range	Best For	Performance Level
Burstable (t3)	2nd Gen Intel Xeon	1-4 vCPU	1GB-16GB	Development, testing, bursty workloads	Baseline performance + burst capability
General Purpose (m5, m6)	Intel/AMD Xeon	1-24 vCPU	4GB-384GB	Most production workloads, balanced compute/memory	Consistent performance, good value
Memory Optimized (r5, r6)	Intel/AMD Xeon	2-96 vCPU	16GB-3072GB	Large in-memory datasets, high transaction volume	Highest memory/vCPU ratio, excellent cache performance
I/O Optimized (i3, i4)	Intel/AMD Xeon	2-96 vCPU	16GB-3072GB	High IOPS, ultra-low latency workloads	NVMe SSD storage, lowest latency

Right-Sizing Methodology

RDS Instance Sizing Approach

Step 1: Estimate Baseline Requirements

- Peak connections: Expected maximum concurrent connections
- Working set size: Data that fits in memory (warm cache)
- Transaction throughput: TPS needed at peak
- Data volume: Total dataset size

Step 2: Calculate Minimum Instance Type

- Memory = Working set size × 1.5 (cache margin)
- vCPU = Peak TPS ÷ ~200 (transactions per vCPU)
- Network = Peak throughput ÷ 8 (choose instance with adequate network)

- └─ Storage = Data volume + 25% growth buffer

Step 3: Test and Monitor

- └─ Load test with realistic workload (TPC-B, HammerDB)
- └─ Monitor CloudWatch metrics:
 - └─ CPU Utilization (target 60-70%)
 - └─ Memory Utilization (target 70-80%)
 - └─ Database Connections
 - └─ Read/Write IOPS
 - └─ Read/Write Latency
- └─ Adjust instance size if metrics exceed targets

Step 4: Production Validation

- └─ Deploy to production, monitor actual usage
- └─ Check Enhanced Monitoring dashboard daily
- └─ Review Performance Insights for query analysis
- └─ Schedule monthly capacity planning reviews

Example Sizing (Production Web Application):

Requirements:

- └─ 1000 concurrent connections (MySQL default ~500/vCPU)
- └─ 500 TPS peak (target ~200 TPS/vCPU)
- └─ 100 GB working dataset (500GB with indexes)
- └─ 2 TB total data volume

Calculation:

- └─ vCPU: $500 \text{ TPS} \div 200 \text{ TPS/vCPU} = 2.5 \rightarrow 4 \text{ vCPU minimum}$
- └─ Memory: $500 \text{ GB} \times 1.5 = 750 \text{ GB} \rightarrow 768 \text{ GB (r5.24xlarge)}$
- └─ Connections OK: $1000 < 4 \text{ vCPU} \times 500 = 2000$
- └─ Instance: db.r5.12xlarge (48 vCPU, 384 GB RAM) for headroom

⚠ Critical Sizing Mistake: Over-provisioning by 3-5x is common. Start conservative, monitor metrics, upsize if needed (takes 15-30 min). Under-provisioning is worse (causes outages). Use AWS Right Sizing recommendations after 2+ weeks of production data.

5. Storage, IOPS and I/O Performance

RDS storage options include General Purpose SSD (gp3), Provisioned IOPS SSD (io1/io2), and Magnetic storage (legacy). Storage type significantly impacts database performance and cost. Proper storage configuration ensures consistent performance under load.

RDS Storage Types Comparison

Storage Type	IOPS Range	Throughput	Cost per GB	Best For	Characteristics
GP3 (Current)	3,000-16,000	125-1,000 MB/s	\$0.115/month	Most workloads, good price/performance	Burstable to 16,000 IOPS, decoupled IOPS/throughput
IO1 (Provisioned)	1,000-64,000	500 MB/s	\$0.10/month + \$0.10/IOPS	I/O intensive, need guaranteed IOPS	Consistent performance, costly for high IOPS
IO2 (Latest)	1,000-64,000	1,000 MB/s	\$0.10/month + \$0.05/IOPS	Ultra-high performance, critical workloads	Higher throughput than IO1, lower IOPS cost
Magnetic (Legacy)	100-1,000 (avg)	N/A	\$0.10/month	Legacy applications only, avoid for new	High latency variability, unpredictable performance

IOPS Provisioning Guidelines

IOPS Calculation for Database Workloads

GP3 (Recommended for most):

- └ Base: 3,000 IOPS included free
- └ Can scale: 0-16,000 IOPS independent of storage size
- └ Cost: \$0.115/GB + additional IOPS cost if >3,000
- └ Good for: 80% of workloads

Provisioned IOPS (IO1/IO2) - Formula:

$$\text{IOPS} = (\text{Total Database Size in GB} \times \text{Read/Write Ratio}) \times \text{Transactions Per Second}$$

Example:

Database Size: 500 GB
Read/Write Ratio: 70/30 = 2.33
TPS: 1000 (20% write = 200 write TPS)
IOPS needed: 500 × 2.33 × 200 = 233,000 IOPS (use IO1/IO2)

Or use simpler rule:

IOPS = Database Size × IO Intensity Factor
Factor: 10-50 IOPS/GB for OLTP, 2-10 for OLAP

Storage Size Calculations:

- └ Current data: 100 GB

- Indexes: 100 GB × 30% = 30 GB
- Logs (binary logs): 5-10 GB/day × retention days
- Temp tables: 10 GB (overhead)
- Growth buffer: 25% of current
- Total: 100 + 30 + 50 + 10 + 45 = 235 GB → provision 250 GB

Cost Comparison Example (500 GB, 5,000 IOPS needed):

GP3: $500 \times \$0.115 + (5,000 - 3,000) \times \$0.035 = \$57.50 + \$70 = \$127.50/\text{month}$

IO1: $500 \times \$0.10 + 5,000 \times \$0.10 = \$50 + \$500 = \$550/\text{month}$ (4x more expensive)

Recommendation: Use GP3 for most workloads, IO1/IO2 only for specialized cases

6. Backup, Recovery and Disaster Recovery

RDS provides automated daily backups with 35-day retention, continuous incremental backups, and manual snapshots. Backup strategy is critical for disaster recovery and compliance. Understanding RPO, RTO, and recovery procedures ensures minimal downtime on failures.

RDS Backup and Recovery Options

Option	Automated	Retention	Point-in-Time Recovery	Cost	Use Case
Automated Backups	Yes, daily	1-35 days (configurable)	Yes, to any point in retention window	Included in RDS pricing (S3 storage)	Primary recovery mechanism
Manual Snapshots	Manual trigger	Indefinite (until deleted)	No (fixed point)	S3 storage cost	Long-term retention, major version upgrades
Continuous Backup (Aurora)	Automatic	35 days (configurable)	Yes, 1-second granularity	Included in storage cost	Aurora optimal, zero RPO
Export to S3	Manual	As long as S3 retained	No (full database export)	S3 storage + export cost (~\$1/GB)	Long-term archive, data lake

Disaster Recovery Strategies

RDS Disaster Recovery Strategies

Strategy 1: Multi-AZ (Recommended for HA)

RPO: ~0 seconds (synchronous replication)

RTO: 60-120 seconds (automatic failover)

Cost: 2x database cost

Setup:

- └ Enable Multi-AZ at creation or modify
- └ AWS automatically creates standby in different AZ
- └ Synchronous replication maintains zero lag
- └ Automatic failover on primary failure

Recovery: Fully automatic, transparent to application

Limitation: Only within single region

Strategy 2: Read Replicas (Cross-Region DR)

RPO: <1 second (asynchronous)

RTO: Manual promotion + DNS update (~5-10 minutes)

Cost: Read replica instance + data transfer

Setup:

- └ Create read replica in different region
- └ Monitor replication lag (CloudWatch metric)
- └ On disaster, promote to standalone database
- └ Update application DNS/connection string

Recovery: Manual, requires runbook and testing

Limitation: Must manually promote, test failover regularly

Strategy 3: Automated Backups + Point-in-Time Recovery

RPO: Backup interval (typically 1 hour)

RTO: 5-30 minutes (restore from backup)

Cost: S3 backup storage only

Setup:

- └ Enable automated backups (1-35 day retention)
- └ Test restore procedures monthly
- └ Document recovery runbook
- └ Alert on backup failures

Recovery: Manual restore creates new instance

Limitation: Data loss between backup and failure, slow RTO

Recommended Hybrid Approach:

Production:

- └ Multi-AZ for local redundancy (60-120 sec RTO)
- └ Cross-region read replica for DR (manual failover)

Staging:

- └ Read replica of production for testing

Backup Retention Policy:

Daily: Retain 7 days (covers typical issues)
Weekly: Monthly snapshots, retain 12 months
Monthly: Year-end snapshots, retain indefinitely (compliance)

Cost: Typically <10% of RDS cost

7. Security, Encryption and Network

RDS security spans multiple layers: network (VPC, security groups), authentication (username/password, IAM), encryption (at-rest with KMS, in-transit with SSL/TLS), and auditing (CloudTrail, enhanced monitoring). Comprehensive security practices prevent unauthorized access and data breaches.

RDS Security Layers

Layer	Protection Mechanism	Implementation	Risk if Missing
Network	Restrict database access to specific networks/IPs	Private subnets, security groups, network ACLs	Database accessible from internet, data exposure
Authentication	Verify user identity	Master user/password, IAM database auth, domain auth	Unauthorized access, compromised accounts
Authorization	Control what authenticated user can access	Database-level permissions, schema/table grants	Privilege escalation, unauthorized data access
Encryption	Protect data at rest and in transit	KMS for at-rest, SSL/TLS for in-transit, always on	Data exposure in backups/storage, MITM attacks
Auditing	Track who accessed what data and when	CloudTrail for API calls, database audit logs, enhanced monitoring	Undetected breaches, compliance violations

Network Security Configuration

Recommended Network Architecture:

Public Subnets:

- └ Application servers / API Gateway

Private Subnets:

- └ RDS Database (no internet route)
- └ Only accepts traffic from app security group

Security Groups:

- └ App SG: Allow outbound to port 3306 (MySQL)
- └ DB SG: Allow inbound from app SG on port 3306
- └ Deny all other traffic (default deny, allow only needed)

Terraform Example:

```
resource "aws_security_group" "rds" {
  name           = "rds-sg"
  description    = "Security group for RDS database"
  vpc_id         = aws_vpc.main.id

  ingress {
    from_port     = 3306
    to_port       = 3306
    protocol       = "tcp"
    security_groups = [aws_security_group.app.id]
    description    = "Allow MySQL from app servers"
  }

  egress {
    from_port     = 0
    to_port       = 0
    protocol       = "-1"
    cidr_blocks   = ["0.0.0.0/0"]
  }

  tags = {
    Name = "rds-sg"
  }
}
```

SSL/TLS Configuration:

1. Enable SSL requirement:

```
aws rds modify-db-instance \
  --db-instance-identifier mydb \
  --db-parameter-group-name mysql80-ssl \
  --apply-immediately
```

2. Connection string:

```
mysql -h mydb.xxxxx.us-east-1.rds.amazonaws.com \
  -u admin -p \
  --ssl-mode=REQUIRED \
  --ssl-ca=/path/to/rds-ca-2019-root.pem
```

Encryption Configuration:

At Rest:

- └ Enable KMS encryption at creation (can't enable after)
- └ Use AWS managed key or custom key
- └ Automatically applied to backups

In Transit:

- └ SSL/TLS enabled by default (enforce in parameter group)
- └ All data in transit encrypted

8. High Availability and Multi-AZ Deployment

Multi-AZ deployment provides automatic failover, zero-downtime maintenance, and protection against AZ failures. RDS synchronously replicates data to standby instance in different AZ, ensuring RPO approaching zero. Understanding Multi-AZ behavior and limitations is critical for production design.

Multi-AZ Architecture and Benefits

Aspect	Single-AZ	Multi-AZ	Impact
Data Loss Risk	High (instance failure = data at risk)	None (synchronous replication to standby)	Multi-AZ: RPO = ~0 seconds
Failover Time	N/A (manual recovery, hours)	60-120 seconds automatic	Multi-AZ: Transparent to application
Maintenance Windows	Database unavailable during patching	Standby patched, then promoted (no downtime)	Multi-AZ: Zero-downtime maintenance
AZ Failure	Database down until manual intervention	Automatic failover to standby in different AZ	Multi-AZ: Survives AZ outage
Cost	Base instance cost	2x instance cost (standby + data replication)	Multi-AZ: ~100% cost increase for production-grade HA

Key Insight: Multi-AZ standby is NOT a read replica. It's a hot standby that cannot be queried, used only for failover. For scaling reads, use read replicas IN ADDITION to Multi-AZ.

9. Read Replicas and Scaling Reads

Read replicas distribute read traffic across multiple database instances, enabling horizontal scaling of read capacity without affecting primary. RDS supports up to 15 read replicas per primary, with asynchronous replication typically <100ms lag. Cross-region replicas enable disaster recovery and reduced latency for global applications.

Read Replica Characteristics and Use Cases

Characteristic	Same-Region	Cross-Region	Implications
Replication Lag	<100ms typical	100ms-1000ms (depends on network)	Cross-region may have stale data
Cost	Replica instance cost	Replica instance + data transfer (~\$0.02/GB)	Cross-region significantly more expensive
Failover	Promote to standalone (seconds)	Promote + DNS change + connection update	Manual process, requires runbook
DR Capability	Not suitable (same AZ risk)	Excellent DR option (different region)	Use cross-region for regional redundancy
Read Performance	Reduces primary load immediately	Also reduces primary load, serves global reads	Both improve read scaling and performance

Read Replica Deployment Strategy

```
# Read Replica Deployment Patterns

Pattern 1: Same-Region Read Replicas (Scaling)
Architecture:
  Primary (db.r5.2xlarge in us-east-1a)
```


- └─ Replica 1 (db.r5.xlarge in us-east-1b) - reporting
- └─ Replica 2 (db.r5.large in us-east-1c) - analytics
- └─ Replica 3 (db.r5.large in us-east-1a) - failover target

Benefits:

- └─ Offload reporting to Replica 1
- └─ Offload analytics to Replica 2
- └─ Failover capability with Replica 3
- └─ Reduced primary CPU/disk I/O

Cost: ~1.5x primary cost (3 replicas)

Pattern 2: Cross-Region Read Replica (Disaster Recovery)

Architecture:

Primary (us-east-1) → Replica (us-west-2)

Benefits:

- └─ Disaster recovery if region fails
- └─ Serves global read traffic from replica
- └─ Can promote to independent database
- └─ Compliance: data in specific regions

Cost: Replica cost + data transfer (~\$0.02/GB)

Pattern 3: Tiered Replicas (Large Scale)

Architecture:

Primary

- └─ Replica 1 (promoted tier 1)
 - └─ Replica 1a (tier 2)
 - └─ Replica 1b (tier 2)
 - └─ Replica 1c (tier 2)

Benefits:

- └─ Reduces primary replication load
- └─ Replicas feed other replicas
- └─ Scales to 100+ read copies
- └─ Multi-tier load distribution

Trade-off: Replication lag increases per tier

Creating Read Replica (AWS CLI):

```
aws rds create-db-instance-read-replica \  
  --db-instance-identifier mydb-replica-1 \  
  --source-db-instance-identifier mydb \  
  --db-instance-class db.r5.xlarge \  
  --availability-zone us-east-1b
```

Creating Cross-Region Replica:

```
aws rds create-db-instance-read-replica \  
  --db-instance-identifier mydb-replica-dr \  
  --source-db-instance-identifier arn:aws:rds:us-east-1:123456789012:db:mydb \  
  --db-instance-class db.r5.xlarge \  
  --region us-west-2
```

Promoting Replica (Failover):

```
aws rds promote-read-replica \  
    --db-instance-identifier mydb-replica-dr  
  
# After promotion:  
# 1. Update application connection string to new endpoint  
# 2. Verify data consistency  
# 3. Monitor new primary's metrics  
# 4. Consider re-creating replica for future DR
```

10. Parameter Groups, Options and Configuration

Parameter groups control database behavior through configuration parameters. Each database engine has unique parameters affecting performance, security, and features. Understanding key parameters enables optimization without restarting (dynamic parameters) or with planned restarts (static parameters).

Common RDS Parameter Optimization

Database	Parameter	Purpose	Typical Value	Dynamic
MySQL	innodb_buffer_pool_size	In-memory cache for data/indexes	75% of instance RAM	No (restart required)
MySQL	max_connections	Maximum concurrent connections	1000-10000 (depends on instance)	Yes
MySQL	innodb_log_file_size	Transaction log size	512 MB (8 GB for large)	No (restart required)
PostgreSQL	shared_buffers	In-memory cache	25% of instance RAM	No (restart required)
PostgreSQL	effective_cache_size	Query planner hint for cache	75% of instance RAM	Yes
PostgreSQL	max_connections	Maximum concurrent connections	300-3000 (pool from app)	No (restart required)

11. Monitoring, Logging and Performance Insights

RDS provides comprehensive monitoring through CloudWatch, Enhanced Monitoring, and Performance Insights. These tools enable proactive issue detection, performance troubleshooting, and capacity planning. Understanding key metrics and alerts prevents production issues.

RDS Monitoring Metrics and Alerts

Metric	Unit	Healthy Range	Alert Threshold	Action When Exceeded
CPU Utilization	%	20-60%	> 75% sustained	Investigate slow queries, scale instance
Database Connections	Count	< 50% max_connections	> 80% max_connections	Check for connection leaks, adjust pool
Read IOPS	Operations/sec	Workload dependent	Sustained at provisioned IOPS	Increase IOPS, optimize queries
Write Latency	Milliseconds	< 5 ms	> 10 ms sustained	Check I/O bottleneck, scale storage
Free Storage Space	GB	> 20% available	< 10% available	Increase allocated storage immediately
Replica Lag	Seconds	< 1 second	> 5 seconds sustained	Check network, scale replica resources

12. Cost Optimization and Analysis

RDS pricing includes instance cost, storage, backups, and data transfer. Significant savings possible through right-sizing, reserved instances, and choosing optimal deployment model. Cost analysis reveals optimization opportunities.

RDS Cost Optimization Strategies

RDS Cost Optimization Examples

Strategy 1: Right-Sizing

Current: db.r5.4xlarge (16 vCPU, 128GB RAM) = \$3.26/hour

Actual usage: 4 vCPU, 40GB RAM utilization

Optimized: db.r5.xlarge (4 vCPU, 32GB RAM) = \$0.852/hour

Savings: \$2.408/hour × 730 hours/month = \$1,757/month (54% reduction)

Strategy 2: Reserved Instances

On-Demand: db.r5.2xlarge = \$1.63/hour

1-Year RI: \$9,497 upfront + \$0.979/hour

3-Year RI: \$17,147 upfront + \$0.653/hour

Annual cost:

On-Demand: \$1.63 × 730 = \$1,189/year

1-Year RI: \$9,497 + (\$0.979 × 730) = \$10,211/year (save \$1,489 vs on-demand)

3-Year RI: \$17,147 + (\$0.653 × 730) = \$17,623/year (save \$4,667 vs on-demand, 26% savings)

Strategy 3: Consolidate Databases

Before: 5 single-AZ instances (dev, test, staging, prod read, prod standby)

Average: db.t3.medium = \$0.372/hour each

Cost: 5 × \$0.372 × 730 = \$1,358/month

After: 1 Multi-AZ prod instance + shared dev/test instance

Prod: db.r5.2xlarge multi-AZ = 2 × \$1.63 = \$3.26/hour

Dev: db.t3.small = \$0.186/hour

Cost: (\$3.26 + \$0.186) × 730 = \$2,530/month (increase, but better HA)

Recommendation: Consolidate non-prod, Multi-AZ prod only

Strategy 4: Aurora Provisioned vs Serverless

Aurora MySQL Provisioned:

└─ 2 db.r5.xlarge instances: 2 × \$0.852 = \$1.704/hour

└─ Storage: 100GB × \$0.20/GB/month = \$20/month

└─ Total: \$1.704 × 730 + \$20 = \$1,244/month

Aurora Serverless:

└─ 2 ACU baseline = 2 × \$0.06 × 730 hours = \$87.60/month

└─ Storage: 100GB × \$0.20/GB/month = \$20/month

└─ Total: \$107.60/month (91% cheaper for variable workloads)

Strategy 5: Multi-AZ Backup Optimization

Before: Multi-AZ + manual snapshots

Snapshot storage: 10 × 50GB = 500GB = ~\$12/month

After: Multi-AZ + automated backups (35 days)

Backup storage: 35 × 50GB = 1.75TB (but includes incremental)

Estimated: 100 GB incremental = ~\$3/month

13. Amazon Aurora Advanced Features

Aurora is AWS's next-generation relational database engine, offering superior performance, availability, and cost compared to traditional engines. Aurora features distributed storage, automatic scaling, serverless option, and global database capabilities.

Aurora Serverless Benefits and Configuration

Feature	Aurora Provisioned	Aurora Serverless	Best Use Case
Scaling	Manual or auto-scaling groups	Automatic, seamless within seconds	Variable, unpredictable workloads
Minimum Cost	1 instance always running	Pause after 15 min inactivity	Dev/test, infrequent usage
Connection Pooling	Application manages	Built-in at proxy layer	Lambda functions (no persistent connections)
Performance	Consistent, predictable	Slight delay during scaling	Consistent workload → Provisioned

14. Production Best Practices

Production RDS deployments require comprehensive strategies for high availability, security, monitoring, and optimization. Following best practices ensures reliable, performant, and cost-efficient database infrastructure.

RDS Production Best Practices Checklist

- **Always Multi-AZ:** Production databases must have Multi-AZ enabled. RPO approaches zero, RTO <2 minutes. Cost doubles but essential for HA.
- **Read Replicas:** Use read replicas to scale read traffic, offload reporting. Consider cross-region replica for DR.
- **Connection Pooling:** Use connection pools (ProxySQL, PgBouncer) to limit connections, improve performance. Never let applications create individual connections.
- **Parameter Groups:** Create custom parameter groups per environment. Version control parameters, document changes, test in staging.
- **Subnet Groups:** Ensure Multi-AZ subnets configured. RDS automatically places standby in different AZ.
- **Option Groups:** For enterprise engines (Oracle, SQL Server), configure appropriate options for features needed.

Production Deployment Checklist

Category	Pre-Deployment	Deployment	Post-Deployment
Architecture	✓ HA design reviewed ✓ DR strategy defined ✓ Capacity planned ✓ IaC prepared	✓ Multi-AZ enabled ✓ Parameter groups applied ✓ Backups configured ✓ Monitoring enabled	✓ Read replicas tested ✓ Failover tested ✓ Backup integrity verified ✓ Performance baseline captured
Security	✓ Network design approved ✓ Encryption enabled ✓ Auth method chosen	✓ Security group configured ✓ SSL/TLS enabled ✓ IAM auth configured	✓ Connection verified encrypted ✓ Audit logging enabled ✓ Penetration test passed
Monitoring	✓ Alerts designed ✓ Dashboards planned ✓ On-call setup	✓ CloudWatch alarms created ✓ Enhanced Monitoring enabled ✓ Performance Insights enabled	✓ Alarms tested ✓ Runbooks created ✓ Team trained
Capacity	✓ Load test completed ✓ Instance size confirmed ✓ Storage size calculated	✓ Monitoring metrics collected ✓ Performance baselines captured	✓ Monthly capacity reviews scheduled ✓ Growth projections tracked

15. Troubleshooting Guide

RDS troubleshooting requires systematic diagnosis of performance issues, connectivity problems, and operational concerns. Understanding common issues and diagnostic procedures enables quick resolution and minimal downtime.

Common RDS Issues and Solutions

Issue	Symptoms	Root Cause	Diagnostic Steps	Solution
Connection Refused	"Connection timed out", "Host unreachable"	Network blocked, database not accessible	1) Check security group rules 2) Verify subnet configuration 3) Test with nc/telnet 4) Check RDS status	1) Add security group rule for port 2) Verify DB in right subnet 3) Check Multi-AZ configuration 4) Restart instance if hung
Slow Queries	Queries taking 10+ seconds, timeouts	Missing indexes, inefficient query, insufficient resources	1) Enable slow query log 2) Check Performance Insights 3) Analyze query execution plan 4) Check CPU/memory metrics	1) Add missing indexes 2) Rewrite inefficient queries 3) Scale instance if resource-limited 4) Partition large tables
High CPU	CPU >80%, slow response times	Resource saturation, inefficient queries	1) Check CloudWatch CPU metric 2) Review Performance Insights 3) Identify top queries 4) Check for locks	1) Scale to larger instance 2) Optimize identified queries 3) Add indexes 4) Implement query caching
Storage Full	"Disk full" error, writes fail	Logs grew large, incorrect table configs	1) Check free storage metric 2) Find large tables	1) Increase allocated storage 2) Archive old data

			3) Check binary log size	3) Truncate logs
			4) Review backup size	4) Clean up snapshots
Failover Slow	Multi-AZ failover takes >2 minutes	Large transactions, recovery process slow	1) Check logs for recovery details 2) Monitor failover metrics 3) Verify standby health	1) Complete long transactions before maintenance 2) Use smaller transactions 3) Review instance size for recovery speed
High Replica Lag	Replica lags primary by >10 seconds	Heavy write load, network latency, replica underprovisioned	1) Check replication lag metric 2) Monitor replica CPU/memory 3) Check network performance	1) Scale replica instance 2) Reduce primary write load 3) Check network path 4) Use same instance size as primary

16. Terminology Reference

Essential RDS and database terminology for clear communication about database architecture, operations, and optimization.

DB Instance

Single RDS database server running one of the supported database engines. Includes compute (vCPU, RAM) and storage (EBS volume). Identified by endpoint DNS name.

Multi-AZ

Deployment configuration with primary database and synchronous standby replica in different AZ. Provides automatic failover (60-120 sec) and zero data loss (RPO ≈ 0 seconds).

Read Replica

Read-only copy of database, asynchronously replicated from primary. Used to scale read traffic, offload reporting, and provide disaster recovery. Can be in same or different region.

Parameter Group

Template of configuration parameters for a database engine. Controls behavior like buffer pool size, max connections, logging level. Each database instance uses one parameter group.

DB Cluster (Aurora Only)

Group of Aurora database instances sharing storage layer. Includes primary and read replicas. Automatic failover within cluster. Distributed storage provides high durability.

RPO (Recovery Point Objective)

Maximum acceptable data loss in case of failure. Multi-AZ RDS has RPO ≈ 0 (synchronous replication). Read replicas have RPO of replication lag (<1 second typically).

RTO (Recovery Time Objective)

Maximum acceptable downtime. Multi-AZ RDS RTO = 60-120 seconds (automatic failover). Read replica manual failover = 5-10 minutes (includes DNS update).

Backup Window

Daily time window when automated backup occurs (typically 8 hours). I/O slight affected during backup. Configurable per instance to avoid business hours.

Maintenance Window

Weekly time window for AWS to patch and maintain RDS. Single-AZ causes downtime. Multi-AZ applies patches to standby first, then promotes to minimize downtime.

IOPS (Input/Output Per Second)

Measure of storage performance. Higher IOPS = faster database operations. Provisioned IOPS SSD lets you specify exact IOPS. GP3 provides good performance at lower cost.

Aurora Serverless

Serverless option for Aurora where AWS automatically manages database capacity scaling. Scales based on CPU utilization. Pay only for capacity you use.

Performance Insights

AWS tool showing database performance metrics in real-time. Identifies slow queries, locks, and resource bottlenecks. Premium feature, additional cost.

Enhanced Monitoring

More detailed OS-level metrics sent to CloudWatch. Includes CPU breakdown, memory, disk I/O, swap usage. Additional cost for sub-minute granularity.

Replication Lag

Delay between write on primary and visibility on replica. Measured in milliseconds or seconds. Indicates replication health. High lag suggests primary overloaded or network congested.

Database Endpoint

DNS name for connecting to RDS instance. Format: {instance-name}.{random-string}.{region}.rds.amazonaws.com. Applications use endpoint instead of IP address.

Subnet Group

Collection of VPC subnets (minimum 2) where RDS instances can be deployed. Must span multiple AZs for Multi-AZ deployment.

Security Group

Virtual firewall controlling inbound/outbound traffic to RDS instance. Default denies all; explicitly allow ports for database access.

KMS Encryption

AWS Key Management Service encryption for RDS. Encrypts data at rest. Must be enabled at instance creation; cannot be added later. Minimal performance impact.

IAM Database Authentication

Use IAM roles/users instead of database username/password for authentication. More secure, no password to manage.
Requires SSL connection.

AWS RDS Masterclass: Complete Guide to Managed Relational Databases

Master database engines, high availability, performance optimization, security, and production best practices for enterprise RDS deployments

© 2025 EngiDock. All Rights Reserved.

For more courses and learning resources, visit www.engidock.com

Bonus: Database Migration Strategies

Migrating on-premises databases to RDS requires careful planning, minimal downtime, and data validation. AWS provides multiple migration tools and strategies depending on database engine, size, and downtime tolerance.

Migration Methods Comparison

Method	Downtime	Data Loss Risk	Complexity	Best For	Tools
Full Load + CDC	Seconds (cutover only)	None (CDC captures changes)	High (setup DMS task)	Large databases, near-zero downtime required	AWS DMS, Continuous replication
Dump/Restore	1-24 hours (depends on size)	None (full snapshot)	Low (mysqldump, pg_dump)	Small-medium databases, acceptable downtime	mysqldump, pg_dump, SQLSERVER backup

Binary Log Replication	Minutes (switch over)	Low (binary log based)	Medium (setup replication)	MySQL/MariaDB, acceptable brief outage	MySQL native replication, DMS
Snapshot Import	No downtime	Data stale by snapshot age	Medium (create snapshot, import)	Test environments, non-critical systems	Database native snapshots + import

Zero-Downtime Migration Architecture

Zero-Downtime Migration Steps (AWS DMS)

Phase 1: Setup (Days 1-2)

1. Create DMS replication instance (dms.c5.xlarge minimum)
2. Create source endpoint (on-premises database)
3. Create target endpoint (RDS instance)
4. Create DMS task with Full Load + CDC
5. Run test migration on non-prod data

Phase 2: Full Load (Day 3)

1. Start DMS full load task
2. Monitor progress (ETA based on data size)
3. Validate row counts match
4. Verify data integrity (checksums)
5. Monitor source database (minimal impact expected)

Phase 3: CDC (Continuous Change Data Capture)

1. DMS begins capturing source changes
2. Replication lag monitored (target alert if >10 seconds)
3. Run daily validation queries
4. Fix any data discrepancies
5. Keep running for 24-48 hours (confidence period)

Phase 4: Cutover (Minutes)

- Step 1: Stop application writes (maintenance mode)
- Step 2: Wait for CDC lag to reach zero (verify in DMS console)
- Step 3: Validate final data match (row counts, checksums)
- Step 4: Update application connection string to RDS
- Step 5: Resume applications
- Step 6: Monitor RDS metrics (CPU, connections, errors)

Rollback plan: If issues detected, revert connection string to on-prem

Phase 5: Post-Migration (Days 4-7)

1. Monitor RDS metrics 24/7 for 1 week
2. Validate application functionality
3. Performance tune if needed
4. Keep on-premises database online for 1 week (safety)

Bonus: Advanced Performance Tuning

RDS performance optimization requires understanding query execution, index strategies, and resource allocation. Advanced tuning can improve response times by 10-100x and reduce costs by spreading load across replicas.

Performance Tuning Methodology

Phase	Activity	Tools	Expected Improvement
Phase 1: Baseline	Capture current performance metrics, slow queries, resource usage	Performance Insights, slow query log, explain plan	Establish starting point (no improvement yet)
Phase 2: Identify Issues	Find top 10 slow queries, missing indexes, table scans	Performance Insights, query profiling, execution plans	Prioritize optimization opportunities (ranked by impact)
Phase 3: Index Optimization	Add missing indexes, remove unused indexes, optimize joins	EXPLAIN ANALYZE, index usage stats, index advisor	5-50x improvement for indexed queries
Phase 4: Query Optimization	Rewrite inefficient queries, use proper joins, reduce result sets	Query profiling, explain plans, query rewrites	10-100x improvement for poorly written queries
Phase 5: Resource Scaling	Scale CPU/memory if resource-limited, use read replicas for read offload	CloudWatch metrics, Performance Insights, load testing	2-10x improvement if resource-bound
Phase 6: Architectural Changes	Partitioning large tables, denormalization where appropriate, caching layer	Database design review, application architecture assessment	Variable (depends on application changes required)

Index Strategy for Database Performance

Index Optimization Strategy

Index Creation Guidelines:

- └ Primary Key: Always on unique identifier (automatic)
- └ Foreign Keys: Index both sides for joins
- └ WHERE Clauses: Index columns in WHERE for quick filtering
- └ JOIN Columns: Index both table's join columns
- └ ORDER BY: Index columns in same order as query
- └ Composite Indexes: (col1, col2, col3) for multi-column filtering
- └ Covering Indexes: Include all columns needed for query (avoid table lookup)

Index Anti-Patterns (Avoid):

- └ Indexing every column (slows writes, uses storage)
- └ Duplicate indexes (redundant overhead)
- └ Indexes on low cardinality columns (M/F, yes/no - full scans faster)
- └ Indexes never used (monitor with sys.dm_db_index_usage_stats)
- └ Missing statistics (query planner makes bad decisions)

Index Maintenance:

Daily:

- └ Monitor slow queries for missing indexes
- └ Check index fragmentation
- └ Review index usage

Weekly:

- └ Rebuild fragmented indexes (>30% fragmented)
- └ Reorganize mildly fragmented indexes (10-30%)
- └ Update statistics

Quarterly:

- └ Review all indexes for usage
- └ Remove unused indexes
- └ Consolidate overlapping indexes

MySQL Index Example:

```
-- Analyze current indexes
SELECT * FROM INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_NAME = 'orders';

-- Create composite index for common query pattern
CREATE INDEX idx_user_date ON orders(user_id, created_at DESC);

-- Verify index usage
EXPLAIN SELECT COUNT(*) FROM orders
WHERE user_id = 123 AND created_at > '2025-01-01';

-- Remove unused index
DROP INDEX idx_old_unused ON orders;
```

Bonus: Security Hardening for RDS

Comprehensive RDS security requires defense-in-depth across network, authentication, authorization, and encryption. Regular security audits and compliance checks prevent unauthorized access and data breaches.

RDS Security Hardening Checklist

Layer	Control	Implementation	Verification
Network	Private deployment, restrict access	Private subnets only, no public IP, security group allows app only	nmap/telnet from public network = no connection
Network	VPC Endpoint for private AWS services	S3 Gateway Endpoint, Secrets Manager Endpoint	All API calls through VPC, no internet gateway
Encryption	At-rest encryption with KMS	Enable at instance creation, use customer-managed key	Describe-db-instances shows Encrypted: true
Encryption	In-transit encryption (SSL/TLS)	Enable SSL requirement in parameter group, enforce in security group	mysql -h ... --ssl-mode=REQUIRED succeeds, without fails
Authentication	Strong password policy	Master user password 16+ chars, mixed case/numbers/symbols	Store in Secrets Manager, rotate every 90 days
Authentication	IAM Database Authentication	Create IAM users, create database users mapped to IAM, use token auth	Applications authenticate via SigV4, no hardcoded passwords
Authorization	Least privilege database users	Create read-only user for reporting, limited user for app	Test: read-only user cannot INSERT/UPDATE
Auditing	CloudTrail for API calls	Enable CloudTrail, log modifications to RDS	Review CloudTrail logs for any RDS changes
Auditing	Database audit logging	Enable database-specific audit (Oracle Audit, SQL Server Trace)	Review logs for unauthorized access attempts
Backup Security	Encrypted backups	Backups use same KMS key as database	Verify backup encryption in RDS console

Bonus: Aurora Global Database

Aurora Global Database replicates database to other regions for disaster recovery, low-latency reads, and global failover. Designed for globally distributed applications with read access worldwide.

Aurora Global Database Architecture

Component	Description	Latency	Cost
Primary Region	Read/write master, all writes occur here	N/A (source of truth)	Full instance + storage costs
Secondary Region	Read-only replica, asynchronous replication from primary	<1 second (typical)	Read-only instance cost + data transfer (\$0.02/GB)
Replication	Physical log replication from primary to secondary	Sub-second for replicated writes to appear	Included (no separate charge)
Failover	Promote secondary to primary (~1 minute)	Near-zero RPO, ~60 second RTO	No additional cost, changed read-only to read-write

Global Database Use Cases

Aurora Global Database Patterns

Use Case 1: Disaster Recovery

Primary: us-east-1 (write, main application)
Secondary: eu-west-1 (read, backup + local reads)

Benefits:

- Automatic failover if primary region fails
- RPO < 1 second (minimal data loss)
- RTO ~60 seconds (automatic promotion)
- No data re-sync needed (full copy maintained)

Use Case 2: Global Read Distribution

Primary: us-east-1 (centralized writes, billing system)
Secondary 1: eu-west-1 (European users read from here)
Secondary 2: ap-southeast-1 (Asian users read from here)

- Benefits:
- └ Sub-100ms latency for reads from closest region
 - └ Single writes (ACID still maintained)
 - └ Automatic replication to all regions
 - └ Cost: Primary + replicas + inter-region data transfer

Use Case 3: Read Separation

Primary: us-east-1 (main application writes)
Secondary: us-west-2 (reporting/analytics reads)

- Benefits:
- └ Reporting doesn't affect primary performance
 - └ Separate scaling for read workload
 - └ Can pause/resume secondary for cost saving
 - └ Much cheaper than separate RDS instance

Global Database Failover (Manual):

```
# Promote secondary to primary
aws rds modify-db-cluster \
  --db-cluster-identifier secondary-cluster \
  --enable-http-endpoint \
  --region eu-west-1

# Update application to use new primary endpoint
# Old primary becomes read-only (can be recovered later)
```

Bonus: RDS vs Aurora Cost Analysis

Detailed cost comparison between RDS open-source engines and Aurora reveals significant savings at scale. Understanding pricing models enables choosing right database for budget constraints.

Comprehensive Cost Comparison

Scenario	RDS MySQL	Aurora MySQL	Aurora Serverless	Winner
Development/Test (small)	db.t3.micro = \$0.017/hr = \$12.40/month	1 ACU baseline = \$0.06/hr = \$43.80/month	2 ACU baseline = \$0.12/hr = \$87.60/month + pause = effective \$30/month	RDS (\$12)

Production (1TB, consistent 100 TPS)	db.r5.2xlarge = \$1.63/hr × 2 (HA) = \$2,378/month + storage = ~\$2,600/month	4 ACU × 2 = 8 ACU = 0.24 × \$0.06/hr × 730 = \$1,051 + storage \$200 = ~\$1,250/month	2-10 ACU autoscaling = ~\$900/month + storage \$200 = ~\$1,100/month	Aurora/Serverless (50% savings)
High Growth (5TB, 500 TPS, 10 read replicas)	Primary + Standby: db.r5.4xlarge × 2 = \$3.26 × 2 = \$4,761/month 10 replicas: \$0.85 × 10 = \$8,500/month Total: \$13,261 + storage = ~\$15,000/month	Primary + 10 read replicas: 16 ACU × 0.06 = \$0.96/hr = \$700 + storage \$1,000 = ~\$1,700/month	Auto-scaling 4-16 ACU = ~\$1,200/month + storage \$1,000 = ~\$2,200/month	Aurora (87% savings)

Bonus: Operational Excellence and Runbooks

Operational excellence requires documented procedures for common tasks, incident response, and maintenance. Runbooks enable quick, consistent responses to events and reduce mean time to recovery.

Essential RDS Runbooks

Incident Type	Initial Response (5 min)	Diagnosis (10 min)	Resolution (30 min)	Post-Incident (Follow-up)
High CPU (>80%)	1. Page on-call 2. Check CloudWatch CPU metric 3. Notify stakeholders	1. Enable Performance Insights 2. Identify top SQL queries 3. Check for locks/waits 4. Monitor trend (increasing?)	1. Scale instance (if sustainable increase) 2. Kill long-running queries 3. Optimize top query 4. Add missing index 5. Implement query cache	1. RCA: Why did CPU spike? 2. Implement long-term fix 3. Add CloudWatch alarm 4. Update runbook
Database Connectivity Lost	1. Confirm not network issue (ping RDS subnet) 2. Check RDS status (console) 3. Alert application teams 4.	1. Check RDS status page 2. Review CloudTrail for recent changes 3. Check security group rules	1. Verify security group allows traffic 2. Check network ACLs 3. Reboot instance if hung 4. Restore from snapshot if corrupt 5.	1. RCA: What caused failure? 2. Implement preventative measures 3. Update monitoring 4. Document root cause

	Trigger communication plan	4. Test manual connection attempt	Failover Multi-AZ if hardware failure
Storage Full (90%+)	1. Declare incident 2. Page database team 3. Prevent new writes if <5% free 4. Notify stakeholders	1. Check free storage metric 2. Identify large tables 3. Check binary log size 4. Review backup size	1. Immediately increase allocated storage 2. Archive old data (if possible) 3. Clean up binary logs 4. Monitor for continued growth 5. Increase allocation by 50% to prevent recurrence 1. RCA: Why did storage grow? 2. Implement retention policies 3. Set up low storage alerts 4. Plan capacity for next quarter
Read Replica Lag >10 sec	1. Check replication lag metric 2. Notify analytics/reporting teams 3. Consider failover if >1 min	1. Monitor primary CPU/memory 2. Check network performance 3. Verify replica resources 4. Check binary log file size	1. Scale replica instance 2. Reduce primary write load 3. Disable temporary reporting queries 4. Check network path 5. Restart replication if stuck 1. RCA: Primary overloaded or replica underprovisioned? 2. Right-size replica resources 3. Distribute load (sharding if needed) 4. Monitor replication lag continuously

Maintenance Window Procedure

RDS Maintenance Window Runbook

Pre-Maintenance (Day Before)

- └─ Schedule maintenance window in calendar
- └─ Alert all stakeholders
- └─ Disable automatic scaling during window
- └─ Create snapshot for rollback
- └─ Prepare communication templates
- └─ Brief on-call team

Single-AZ Maintenance (Downtime Expected)

1. Set database to maintenance mode (reject new connections)
2. Wait for existing connections to drain (or kill after timeout)
3. AWS patches instance (30-60 min downtime)
4. Instance reboots
5. Verify connectivity and data integrity
6. Resume normal operations

Multi-AZ Maintenance (Minimal Downtime)

1. AWS patches standby instance (no impact to primary)

2. AWS fails over to standby (60-120 sec downtime)

3. Original primary becomes new standby

4. AWS patches original primary

5. Resume normal operation (both updated)

Post-Maintenance Validation

- └ Verify application connectivity
- └ Check database status (green)
- └ Monitor CPU/memory/connections
- └ Run smoke tests
- └ Verify replica sync (if applicable)
- └ Check error logs for issues
- └ Send all-clear notification

Rollback Procedure

If critical issues detected:

1. Restore from pre-maintenance snapshot
2. Update endpoints if using new instance
3. Accept data loss since last backup
4. Contact AWS support for investigation

Bonus: RDS Automation and Infrastructure as Code

Automating RDS provisioning through Terraform enables consistent, repeatable deployments. Infrastructure as Code documents intended state and enables version control for database infrastructure.

Terraform RDS Module Example

```
# Terraform - Comprehensive RDS MySQL Deployment

resource "aws_db_instance" "production" {
  identifier      = "myapp-prod-mysql"
  engine          = "mysql"
  engine_version  = "8.0.33"
  instance_class  = "db.r5.2xlarge"
  allocated_storage = 500
  storage_type    = "gp3"
  iops            = 10000

  # High Availability
  multi_az          = true
  db_subnet_group_name = aws_db_subnet_group.main.name
  publicly_accessible = false
  vpc_security_group_ids = [aws_security_group.rds.id]
```

```
# Security
storage_encrypted      = true
kms_key_id             = aws_kms_key.rds.arn

# Backup
backup_retention_period = 35
backup_window           = "03:00-04:00"
copy_tags_to_snapshot  = true

# Maintenance
maintenance_window     = "sun:04:00-sun:05:00"
auto_minor_version_upgrade = true

# Parameter Group
parameter_group_name    = aws_db_parameter_group.mysql80.name

# Deletion Protection
deletion_protection     = true
skip_final_snapshot     = false
final_snapshot_identifier = "myapp-prod-final-snapshot"

# Monitoring
enabled_cloudwatch_logs_exports = ["error", "general", "slowquery"]
monitoring_interval             = 60
monitoring_role_arn             = aws_iam_role.rds_monitoring.arn

tags = {
    Name          = "myapp-prod-mysql"
    Environment   = "production"
    Team          = "platform"
    CostCenter    = "engineering"
}

# Read Replica for offloading reads
resource "aws_db_instance" "read_replica" {
    identifier          = "myapp-prod-mysql-read-1"
    replicate_source_db = aws_db_instance.production.identifier
    instance_class      = "db.r5.xlarge"

    publicly_accessible = false

    tags = {
        Name = "myapp-prod-mysql-read-1"
    }
}

# Parameter Group with optimized settings
resource "aws_db_parameter_group" "mysql80" {
    name     = "myapp-mysql80"
    family   = "mysql8.0"
```

```

parameter {
    name = "character_set_server"
    value = "utf8mb4"
}

parameter {
    name = "max_connections"
    value = 5000
}

parameter {
    name = "innodb_buffer_pool_size"
    value = "{DBInstanceClassMemory*3/4}"
}

tags = {
    Name = "myapp-mysql80"
}

```

Bonus: Compliance and Regulatory Requirements

AWS RDS provides features for compliance with regulations like HIPAA, PCI-DSS, and SOC 2. Understanding compliance controls and best practices enables configuring RDS appropriately for regulated environments.

RDS Compliance Features and Controls

Regulation	Key Requirements	RDS Controls	Additional Steps
HIPAA	Data encryption, audit logs, access controls, breach notification	KMS encryption, CloudTrail, IAM database auth, VPC isolation	Configure HIPAA BAA with AWS, enable Enhanced Monitoring, implement DLP policies
PCI-DSS	Encryption, network isolation, access logging, vulnerability scanning	KMS encryption, private deployment, VPC Flow Logs, database audit logs	Regular patching, security group restrictions, SSL/TLS enforcement, quarterly assessments
SOC 2	Access controls, encryption, monitoring, incident response, change management	IAM policies, KMS, CloudWatch alarms, CloudTrail, parameter group versioning	Document procedures, implement change control, maintain audit trails, annual certification

GDPR

Data residency, right to be forgotten, data portability, consent

Region-specific deployment, automated backups for export, encryption

Data residency documentation, deletion procedures for user data, DPA with AWS

Engidock